
TECHNICAL BLUEPRINT

Superboard AI Whiteboard

Complete developer specification for building a production-grade K-12 AI-powered collaborative tutoring platform. Covers all features, APIs, database schema, and deployment infrastructure.

FRAMEWORK

Next.js 16 + TypeScript

DATABASE

PostgreSQL (Prisma 6)

API ENDPOINTS

50+ REST endpoints

REAL-TIME

Yjs CRDT + Hocuspocus

AI ENGINE

Anthropic Claude 3

COMPONENTS

75+ React components



Table of Contents

1. Executive Overview	4
1.1 Key Technical Metrics	4
2. Technology Stack	5
2.1 Core Framework	5
2.2 Canvas and Collaboration Libraries	5
2.3 AI and Intelligence	6
2.4 Video and Communication	6
2.5 Authentication and Security	6
2.6 Payments	7
2.7 State Management and Data	7
2.8 UI Component Libraries	7
2.9 Other Key Dependencies	8
3. Database Schema	8
3.1 Enums	8
3.2 Core Models	9
3.2.1 User Model	9
3.2.2 Student Model	10
3.2.3 Room Model	10
3.2.4 Other Models Summary	11
4. API Endpoints	12
4.1 Authentication Endpoints	12
4.2 Room Management Endpoints	12
4.3 AI Endpoints	13
4.4 Video Endpoints	13
4.5 Billing Endpoints	14
4.6 Agency Endpoints	14
4.7 Admin Endpoints	14
4.8 Content and Learning Endpoints	15

5. Authentication and Security	16
5.1 Authentication Flow	17
5.2 CSRF Protection	17
5.3 Rate Limiting	17
5.4 Content Security Policy	18
5.5 Role-Based Access Control	18
5.6 Input Validation	18
5.7 Prompt Injection Prevention	19
6. Real-Time Collaboration Architecture	19
6.1 Synchronized State	19
6.2 Hocuspocus Server Configuration	20
6.3 Hocuspocus Authentication	20
6.4 Client-Side Integration	20
7. AI System	20
7.1 Model Routing	21
7.2 Text AI Actions (Claude 3 Haiku)	21
7.3 Vision AI Actions (Claude 3.5 Sonnet)	22
7.4 Credit System	22
7.5 Prompt Sanitization	22
8. Pricing and Tier System	22
8.1 Pricing Plans	23
8.2 Usage Limits	23
8.3 Feature Gating Matrix	23
8.4 Credit Packs	24
9. UI Components Inventory	24
9.1 Canvas and Whiteboard Components	24
9.2 AI Feature Components	25
9.3 Dashboard Components	25
9.4 Other Key Components	26
10. Custom Hooks	27
11. Library Modules	27

12. Page Routes	28
13. Deployment Infrastructure	29
13.1 Docker Stack	29
13.2 Dockerfile (Multi-Stage)	29
13.3 Environment Variables	30
14. Webhook System	30
15. Audit System	31
16. PWA Support and Branding	31
17. Data Seeding Scripts	31
18. Key Architectural Patterns	32

1. Executive Overview

Superboard is a production-grade, full-stack K-12 AI-powered collaborative tutoring whiteboard web application. It enables real-time multi-user drawing collaboration, AI-powered teaching tools (quiz generation, worksheet creation, graph plotting, handwriting recognition), built-in video calling, lesson scheduling, homework management, agency and multi-tutor support, and Stripe-based billing. The platform is designed for the online tutoring market with a focus on real-time collaboration, educational AI tools, and scalable multi-tenant architecture.

The application serves two primary user types: **Tutors** (individual educators or agency owners who create lesson rooms, draw on the whiteboard, and use AI tools) and **Students** (learners who join rooms, participate in real-time collaboration, and complete homework). The platform supports an agency model where agency owners can invite sub-tutors, manage student rosters, brand the experience with custom logos and colors, and track analytics across their organization.

This technical blueprint provides a complete specification of every feature, API endpoint, database model, authentication mechanism, real-time collaboration architecture, AI integration, payment system, and deployment infrastructure. A development team can use this document to independently build the entire application from scratch without needing access to the original codebase.

1.1 Key Technical Metrics

Metric	Value
Application Name	Superboard
Framework	Next.js 16 (App Router)
Language	TypeScript
UI Library	React 19 + Tailwind CSS 4 + shadcn/ui
Database	PostgreSQL (Supabase)
ORM	Prisma 6
Real-Time Engine	Yjs CRDT + Hocuspocus
Canvas Library	Fabric.js 6
AI Provider	Anthropic Claude 3 (Haiku + Sonnet)
Video	LiveKit (self-hosted)
Payments	Stripe
Auth	Supabase Auth (JWT)
API Endpoints	50+

Metric	Value
Database Models	20+
UI Components	75+
AI Actions	24
Custom Hooks	9
Library Modules	20+
Pricing Tiers	4 (FREE, PRO, AGENCY_STANDARD, AGENCY_PREMIUM)

2. Technology Stack

The technology stack has been carefully chosen to balance developer productivity, performance, scalability, and maintainability. Each layer of the application uses industry-standard tools that are well-documented and have large community support, ensuring that any development team can work with familiar technologies.

2.1 Core Framework

Technology	Version	Purpose
Next.js	^16.1.1	React framework with App Router, React Server Components, and API routes
React	^19.0.0	UI component library for building interactive user interfaces
TypeScript	^5	Statically typed JavaScript for improved code quality and IDE support
Tailwind CSS	^4	Utility-first CSS framework for rapid UI styling
Prisma	^6.19.2 / ^6.11.1	Type-safe ORM for database access with schema-first approach
PostgreSQL (Supabase)	N/A	Primary relational database with managed hosting
Bun	N/A	Fast JavaScript runtime for package management and production server

2.2 Canvas and Collaboration Libraries

The real-time whiteboard is built on a CRDT (Conflict-free Replicated Data Type) architecture using Yjs, which ensures that multiple users can draw and edit simultaneously without conflicts. Each canvas object is individually synchronized, allowing for fine-grained collaboration even on slow networks. The Hocuspocus server acts as the WebSocket relay between clients, managing document state and authentication.

Library	Purpose
Fabric.js 6	Interactive HTML5 canvas for drawing shapes, freehand lines, text, and images
Yjs	CRDT library that enables conflict-free real-time collaboration across multiple clients
@hocuspocus/provider ^4.5.0	Client-side Yjs WebSocket provider connecting to the Hocuspocus server
@hocuspocus/server ^4.4.0	Server-side Yjs sync engine handling connections, auth, and persistence
y-protocols	Yjs awareness protocol for cursor presence and role broadcasting
perfect-freehand	Smooth freehand drawing algorithm for natural pen strokes

2.3 AI and Intelligence

The AI system uses Anthropic Claude models with intelligent routing between two model tiers. Fast, inexpensive text generation tasks are routed to Claude 3 Haiku, while vision-dependent tasks that require image analysis (such as graph plotting from handwritten input or geometric shape recognition) are routed to the more capable Claude 3.5 Sonnet. The system includes prompt sanitization to prevent injection attacks and supports 24 distinct educational AI actions.

Library	Purpose
@anthropic-ai/sdk ^0.115.0	Official Anthropic SDK for Claude API integration
katex ^0.18.1	Fast LaTeX math formula rendering in the browser

2.4 Video and Communication

Library	Purpose
livekit-client ^2.21.0	Real-time video/audio client for browser-based video calls
livekit-server-sdk ^2.17.0	Server-side SDK for generating LiveKit JWT tokens
@livekit/components-react ^2.9.23	Pre-built React components for LiveKit video UI

2.5 Authentication and Security

Library	Purpose
@supabase/ssr	Supabase authentication for server-side rendering
@supabase/supabase-js	Supabase client for browser-side auth and database access

Library	Purpose
Zod ^4.0.2	Type-safe input validation for all API endpoints
@fingerprintjs/fingerprintjs	Browser fingerprinting for anonymous student tracking
react-hook-form + @hookform/resolvers	Form validation with Zod schema integration

2.6 Payments

Stripe handles all payment processing including subscription management, metered billing for agency hourly usage, credit pack purchases, and webhook-driven event processing. The system supports both monthly and annual billing cycles with automatic proration.

Library	Purpose
Stripe ^22.4.0	Payment processing, subscriptions, metered billing, and webhook handling

2.7 State Management and Data

Library	Purpose
Zustand ^5.0.6	Lightweight global client state management
@tanstack/react-query ^5.82.0	Server state caching, synchronization, and background updates
@tanstack/react-table ^8.21.3	Headless UI for building data tables with sorting and pagination

2.8 UI Component Libraries

The application uses shadcn/ui as its primary component library, which provides over 40 accessible, customizable UI components built on top of Radix UI primitives. Additional libraries handle specific UI needs: Framer Motion for animations, recharts for data visualization, react-markdown for rendering markdown content, react-syntax-highlighter for code display, and dnd-kit for drag-and-drop functionality.

Library	Purpose
shadcn/ui (Radix UI)	40+ accessible UI components (dialog, tabs, popover, etc.)
Lucide React	Icon library with 1000+ SVG icons
Framer Motion ^12.23.2	Production-ready animation library for React
cmdk	Command palette component (Cmd+K style search)
Embla Carousel	Carousel/slider component

Library	Purpose
react-resizable-panels	Resizable panel layouts
sonner	Toast notification system
vaul	Drawer (bottom sheet) component
recharts ^2.15.4	Charts and data visualization
react-markdown	Markdown rendering with GFM support
react-syntax-highlighter	Syntax-highlighted code blocks
dnd-kit	Drag and drop functionality

2.9 Other Key Dependencies

Library	Purpose
sharp ^0.35.3	Server-side image processing and compression
date-fns	Date utility functions for formatting and manipulation
uuid ^4.0.1	UUID generation for unique identifiers
pg ^8.23.0	PostgreSQL driver for direct database connections
next-auth ^4.24.11	Authentication library for Google OAuth support

3. Database Schema

The database schema is managed through Prisma ORM with PostgreSQL as the backend (hosted on Supabase). The schema defines 9 enums and 20 models that cover all aspects of the application: user management, room-based lessons, collaboration tracking, agency operations, billing, homework management, content library, and administrative functions. All models use UUID primary keys for global uniqueness and include automatic timestamp tracking via createdAt and updatedAt fields.

3.1 Enums

Enum Name	Values
Tier	FREE, PRO, AGENCY, AGENCY_STANDARD, AGENCY_PREMIUM
UserStatus	ACTIVE, SUSPENDED, BANNED
Subject	MATH, SCIENCE, LANGUAGE, GENERAL, MUSIC, CODING, TEST_PREP, ART, ESL

Enum Name	Values
LessonStatus	SCHEDULED, COMPLETED, CANCELLED, NO_SHOW
InviteStatus	PENDING, ACCEPTED, EXPIRED, CANCELLED
CreditPackStatus	ACTIVE, EXHAUSTED, EXPIRED
HomeworkStatus	PENDING, SUBMITTED, GRADED, OVERDUE
InvoiceStatus	DRAFT, SENT, PAID, OVERDUE, CANCELLED
QuestionType	OPEN, MCQ, TRUE_FALSE, MULTI_SELECT, FILL_IN_BLANK, SHORT_ANSWER, ESSAY, MATCHING, ORDERING

3.2 Core Models

3.2.1 User Model

The User model represents tutors and agency owners. It includes authentication fields (email, managed by Supabase Auth), tier information for feature gating and billing, Stripe integration fields, referral tracking, and agency-specific branding fields. The self-referential relationship via `parentAgencyId` enables the agency hierarchy where agency owners can invite sub-tutors.

Field	Type	Notes
id	UUID PK	Auto-generated
email	String (unique)	User login email
name	String?	Display name
tier	Tier (enum)	Default: FREE
isAdmin	Boolean	Default: false
status	UserStatus (enum)	Default: ACTIVE
stripeCustomerId	String? (unique)	Stripe customer ID
stripeSubscriptionId	String?	Stripe subscription ID
fingerprintHash	String?	Browser fingerprint hash
gracePeriodEndsAt	DateTime?	Grace period for downgraded users
referralCode	String? (unique)	Unique referral code
referredByCode	String?	Referrer code used at signup
referralCount	Int	Default: 0
referralRewardClaimed	Boolean	Whether referral reward was claimed

Field	Type	Notes
monthlyAiBudgetCents	Int	Default: 0; tracks monthly AI spend
customDomain	String? (unique)	Custom domain for white-labeling
brandingLogoUrl	String?	Agency logo URL
brandingColor	String?	Hex color code for branding
agencyName	String?	Agency display name
parentAgencyId	String? (FK User)	Self-referencing agency hierarchy
createdAt	DateTime	Auto-generated
updatedAt	DateTime	Auto-updated

3.2.2 Student Model

The Student model manages the agency student roster. Each student belongs to an agency (via agencyId FK) and includes contact information, grade level, and an optional parent access token for the parent portal feature. The unique constraint on [agencyId, email] prevents duplicate students within the same agency.

Field	Type	Notes
id	UUID PK	Auto-generated
agencyId	FK to User (Cascade)	Owning agency
email	String	Student email
name	String	Student display name
grade	String?	Grade level
phone	String?	Contact phone
parentName	String?	Parent or guardian name
parentEmail	String?	Parent email for notifications
parentAccessToken	String? (unique)	Token for parent portal access
isActive	Boolean	Default: true
notes	Text?	Internal notes

3.2.3 Room Model

The Room model represents a lesson session. Each room is created by a tutor and can have multiple board pages, participants, recordings, and lesson notes. The durationMinutes field is updated when the lesson ends for metered billing. Branding fields allow agency-specific white-labeling per room.

Field	Type	Notes
id	UUID PK	Room identifier
tutorId	FK to User (Cascade)	Creating tutor
subject	Subject (enum)	Default: GENERAL
isActive	Boolean	Whether lesson is in progress
startedAt	DateTime?	Lesson start time
endedAt	DateTime?	Lesson end time
durationMinutes	Int	Default: 0; calculated on end
brandingLogo	String?	Room-specific branding logo
brandingColor	String?	Room-specific branding color

3.2.4 Other Models Summary

The remaining models provide specialized functionality across the platform. Each model is designed with appropriate indexes for query performance and constraints for data integrity. Below is a comprehensive summary of all additional models with their key fields and purposes.

Model	Key Fields	Purpose
BoardPage	roomId, pageIndex, snapshot (Text)	Multi-page canvas snapshots (max 5MB each)
RoomParticipant	roomId, studentId, studentIdentity, studentName, joinedAt	Student join tracking per room
Template	tutorId, name, subject, snapshot	Saved board layouts for reuse
Recording	roomId, tutorId, url, status, duration, egressId	Lesson video recordings
UsageLog	userId, periodStartDate, videoMinutesUsed, aiCreditsUsed	Weekly/monthly usage tracking
AgencyInvite	code (8-char unique), agencyId, invitedEmail, recipientId, status, expiresAt	Sub-tutor invitations with 7-day expiry
Subscription	userId, stripeSubscriptionId, planName, status, periodStart/End	Stripe subscription tracking
AuditLog	adminId, action, targetType, targetId, metadata	Admin action audit trail
ScheduledLesson	tutorId, title, subject, studentEmail, scheduledAt, durationMinutes, status, maxStudents, isGroup	Lesson scheduling with group support
WebhookConfig	userId, url, events[], secret, isActive	User webhook endpoints with HMAC
PlatformConfig	id (fixed "platform"), maintenanceMode, announcementText	Singleton platform configuration

Model	Key Fields	Purpose
CreditPack	agencyId, hoursPurchased, hoursRemaining, pricePaidCents, status	Prepaid hourly credit packs
Homework	agencyId, studentId, tutorId, roomId, title, description, subject, dueDate, status	Student homework assignments
LessonNote	roomId, tutorId, studentId, content, tutorFeedback, topicsForNext, rating	Post-lesson notes (one per room)
ResourceLibrary	agencyId, uploadedByTutorId, name, category, subject, fileUrl, fileType	Shared agency resource library
Invoice	agencyId, studentId, invoiceNumber, amountCents, currency, status, lessonHours	Agency billing invoices
QuestionItem	subject, gradeBand, topic, difficulty, stem, answerKey, questionType, tags	Cloud question bank with standards
TestPrepCategory	name, testType, subject, gradeLevel	Standardized test categories (SAT, ACT)
CurriculumStandard	code, framework, subject, gradeBand, topic, parentCode	Hierarchical curriculum standards

4. API Endpoints

The application exposes over 50 REST API endpoints organized into logical route groups. All endpoints use JSON request/response format, JWT-based authentication via Bearer tokens, and Zod-validated input. Rate limiting is applied per-category via middleware, and CSRF protection is enforced on all state-changing requests. Each endpoint group is described below with its methods, paths, authentication requirements, and core functionality.

4.1 Authentication Endpoints

Method	Route	Auth	Purpose
GET	/api/auth/profile	Optional	Get user profile with tier and name
POST	/api/auth/register	None	Auto-register OAuth users (id, email, name)

4.2 Room Management Endpoints

Room endpoints handle the lifecycle of lesson sessions: creation, retrieval, updating, joining, and template management. Room creation includes tier validation (FREE limited to 1 active room), automatic BoardPage initialization, and branding inheritance from the tutor profile or custom values.

Method	Route	Auth	Purpose
POST	/api/room	Required (tutor)	Create new lesson room with tier check
GET	/api/room	Required	Get room by ID or list by tutor
PATCH	/api/room/[roomId]	Required	Update room (end lesson, etc.)
GET	/api/room/list	Required	List all rooms for current user
POST	/api/room/join	None (student)	Student joins a room by identity
POST	/api/room/participants	Required	Track student participation
GET	/api/room/templates	Required	List saved templates
POST	/api/room/templates	Required	Save a template from current board
GET	/api/room/templates/[id]	Required	Get specific template
DELETE	/api/room/templates/[id]	Required	Delete a template
GET	/api/room/[roomId]/recording	Required	Get recording for room
POST	/api/room/[roomId]/video-heartbeat	Required	Track video usage minutes

4.3 AI Endpoints

The AI system routes requests to either Claude 3 Haiku (for text-based educational content generation) or Claude 3.5 Sonnet (for vision-based analysis of handwritten input, shapes, and graphs). Each action has a credit cost that is deducted from the user monthly or weekly budget, depending on their tier. Enhanced AI actions (lesson plans, rubrics, differentiated instruction, etc.) are exclusive to PRO and Agency tiers.

Method	Route	Purpose
POST	/api/ai/action	Execute AI action (routed to Haiku or Sonnet based on action type)
POST	/api/ai/answer-key	Generate answer key for quiz questions

4.4 Video Endpoints

Method	Route	Auth	Purpose
POST	/api/livekit/token	Required	Generate LiveKit JWT token with video limit check
POST	/api/livekit/webhook	None (verified)	Receive LiveKit webhook events

4.5 Billing Endpoints

Method	Route	Auth	Purpose
GET	/api/stripe/checkout?plan=	Required	Create Stripe checkout session
POST	/api/stripe/webhook	None (verified)	Receive Stripe webhook events

4.6 Agency Endpoints

Agency endpoints manage the multi-tutor organizational structure. Agency owners can invite sub-tutors via unique 8-character codes (valid for 7 days), manage student rosters with bulk import capability, track usage analytics across their organization, and manage prepaid credit packs for hourly billing.

Method	Route	Auth	Purpose
POST	/api/agency/invite	Agency	Create sub-tutor invite (8-char code, 7-day expiry)
GET	/api/agency/invite	Agency	List agency invites (with lazy expiry cleanup)
GET	/api/agency/invite/[code]	None	Accept invite by code
POST	/api/agency/invite/[code]/cancel	Agency	Cancel pending invite
GET	/api/agency/students	Agency	List agency students
POST	/api/agency/students	Agency	Register new student
PATCH	/api/agency/students/[id]	Agency	Update student details
POST	/api/agency/students/import	Agency	Bulk import students
POST	/api/agency/students/register	Agency	Register student for lesson
GET	/api/agency/subtutors/[id]	Agency	Get sub-tutor details
GET	/api/agency/analytics	Agency	Agency usage analytics
GET	/api/agency/hours	Agency	Agency lesson hours tracking
GET	/api/agency/credit-packs	Agency	Get credit packs
POST	/api/agency/credit-packs	Agency	Purchase credit pack

4.7 Admin Endpoints

Admin endpoints provide platform-wide management capabilities including user administration, subscription oversight, room management, billing analytics, audit log access, and platform configuration. All admin endpoints require the caller to have `isAdmin` set to `true`.

Method	Route	Purpose
GET	/api/admin/check	Check if current user is admin
GET	/api/admin/stats	Platform-wide statistics
GET/PATCH	/api/admin/config	Get/update platform configuration (maintenance mode, announcements)
GET	/api/admin/config/public	Get public platform config
GET	/api/admin/users	List all users with filtering
POST	/api/admin/users/bulk	Bulk user operations
PATCH	/api/admin/users/[id]	Update user details
POST	/api/admin/users/[id]/ban	Ban or unban a user
POST	/api/admin/users/[id]/reset-password	Reset user password
GET	/api/admin/users/export	Export users list
GET	/api/admin/subscriptions	List all subscriptions
GET	/api/admin/rooms	List all rooms
POST	/api/admin/rooms/[id]/force-end	Force-end an active room
GET	/api/admin/billing	Admin billing overview
GET	/api/admin/audit	Audit log entries

4.8 Content and Learning Endpoints

Method	Route	Purpose
GET/POST	/api/homework	List/create homework assignments
PATCH	/api/homework/[id]	Update homework (grade, feedback)
GET/POST	/api/lesson-notes	List/create lesson notes
GET	/api/lesson-notes/[roomId]	Get notes for specific room
POST	/api/lesson-notes/batch	Batch create/update notes
GET/POST	/api/recordings	List recordings
GET/POST	/api/schedule	CRUD scheduled lessons
GET/PATCH	/api/schedule/[id]	Get/update lesson
GET/POST	/api/resources	CRUD resource library

Method	Route	Purpose
GET/DELETE	/api/resources/[id]	Get/delete resource
GET/POST	/api/webhooks	CRUD webhook configs
GET/PATCH/DELETE	/api/webhooks/[id]	Manage webhook
GET/POST	/api/invoices	CRUD invoices
GET/PATCH	/api/invoices/[id]	Get/update invoice
GET	/api/questions	Query question bank
GET	/api/questions/[id]	Get specific question
POST	/api/manipulative/generate	AI-generated math manipulatives
GET	/api/test-prep/categories	List test prep categories
POST	/api/test-prep/assign	Assign test prep to student
GET	/api/student/[id]/progress	Student progress data
GET	/api/parent/[token]	Parent portal data via access token
GET	/api/analytics	Usage analytics for current user
GET	/api/calendar/ics/[id]	Generate ICS calendar file
GET	/api/usage/current	Current period usage data
GET	/api/usage/agency	Agency-wide usage data
POST	/api/usage/fingerprint	Report browser fingerprint
GET/POST	/api/referral	Get/generate referral code
POST	/api/referral/apply	Apply referral code
POST	/api/referral/claim	Claim referral reward
GET	/api/health	Health check endpoint

5. Authentication and Security

Security is implemented at multiple layers throughout the application. The authentication system uses Supabase Auth with JWT tokens, supported by middleware-enforced CSRF protection, category-based rate limiting, and comprehensive input validation. Role-based access control (RBAC) ensures that each user type can only access features appropriate to their role.

5.1 Authentication Flow

Authentication is handled by Supabase Auth with support for email/password login and Google OAuth. The client-side AuthGate component checks for an active Supabase session on mount. Unauthenticated users see the LandingPage with login/register modals, while authenticated users are routed to the DashboardPage. OAuth users are automatically registered via the /api/auth/register endpoint. All API routes verify JWT Bearer tokens using the verifyAuth() and requireAuth() helper functions from the auth library module.

The authentication architecture uses a three-tier validation approach. First, the Next.js middleware applies rate limiting to all API routes before they reach the route handlers. Second, CSRF double-submit cookie validation is enforced on all non-idempotent HTTP methods (POST, PUT, PATCH, DELETE), using a constant-time comparison function to prevent timing attacks. Third, each API route handler calls requireAuth() or requireAdmin() to verify the JWT token and extract the user identity.

5.2 CSRF Protection

CSRF protection uses the double-submit cookie pattern implemented in the Next.js middleware. For non-idempotent requests (POST, PUT, PATCH, DELETE), the middleware generates a CSRF token stored as an httpOnly=false, secure, sameSite=lax cookie with a 1-hour maxAge. The same token is echoed in the X-CSRF-Token response header. On subsequent requests, the middleware compares the cookie value with the header value using timingSafeEqualEdge(), a constant-time XOR-based comparison that prevents timing side-channel attacks. Webhook and Stripe endpoints are excluded from CSRF validation to allow server-to-server callbacks.

5.3 Rate Limiting

Rate limiting is applied at the middleware level with per-category limits. The system supports both distributed rate limiting via Upstash Redis (when UPSTASH_REDIS_REST_URL is configured) and an in-memory fallback for development environments. Unknown IP addresses receive a strict 10 requests per minute cap regardless of category.

Category	Max Requests	Window	Applies To
livekit	10	60s	Video token generation
auth	20	60s	Authentication endpoints
ai	30	60s	AI action endpoints
participants	50	60s	Room participant tracking
roomJoin	10	60s	Room join attempts
parentPortal	5	15 min	Parent portal access

Category	Max Requests	Window	Applies To
webhook	20	60s	Webhook management
default	100	60s	All other API routes

5.4 Content Security Policy

The Content Security Policy (CSP) is dynamically generated per request using a cryptographic nonce for script-src directives. The middleware builds the CSP header with directives that allow connections to Supabase, Stripe, LiveKit, Anthropic API, Mathpix API, Google OAuth, and the Hocuspocus WebSocket server. The CSP is applied to all page routes (not API routes, which use a stricter default-src none policy).

Additional security headers applied via next.config.ts include X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy: camera=(self), microphone=(self), geolocation=(), Cross-Origin-Opener-Policy: same-origin, Cross-Origin-Resource-Policy: same-origin, and X-Powered-By is removed to prevent framework fingerprinting.

5.5 Role-Based Access Control

The RBAC system defines five roles (Admin, Agency Owner, Sub Tutor, Tutor, Student) with granular permissions across six categories (Room, Whiteboard, AI, Recording, Agency, Admin). The hasPermission() function checks whether a given role includes a specific permission, and getUserRole() determines the role based on user attributes (isAdmin, tier, parentAgencyId).

Permission	Admin	Agency Owner	Sub Tutor	Tutor	Student
room:create / room:end	Yes	Yes	Yes	Yes	No
whiteboard:draw / tools / pages	Yes	Yes	Yes	Yes	Yes/Yes/No
ai:use / ai:answer_key	Yes	Yes	Yes	Yes	No
recording:start / view	Yes	Yes	Yes	Yes	No
agency:students / subtutors / analytics / branding	Yes	Yes	No	No	No
admin:view / users / platform	Yes	No	No	No	No

5.6 Input Validation

All API endpoints use Zod v4 validation schemas defined centrally in the `validations.ts` module. The `validateInput` helper function parses incoming data against the schema and returns either the validated typed data or a 400 JSON error response with field-level error details. Validation rules include string length limits, regex patterns for identifiers, UUID validation, URL format checking (HTTPS-only for webhooks), and max size limits for binary data (5MB for snapshots and images).

5.7 Prompt Injection Prevention

The AI module (`ai.ts`) implements a `sanitizePrompt()` function that scans user prompts against 10 injection patterns before sending them to the Claude API. Detected patterns include "ignore previous instructions", "you are now a", "jailbreak", "pretend you are not", "output the system prompt", and similar attempts. Matches are replaced with [FILTERED] to neutralize the attack while preserving the rest of the prompt context.

6. Real-Time Collaboration Architecture

The real-time collaboration system is built on Yjs, a CRDT library that enables conflict-free concurrent editing. Unlike Operational Transformation (OT) systems, CRDTs guarantee eventual consistency without requiring a central server to resolve conflicts, making the system resilient to network partitions and latency. The Hocuspocus server acts as the WebSocket relay, managing client connections, authentication, and document state.

The collaboration architecture supports several advanced features beyond basic object synchronization. These include cursor presence via the Yjs awareness protocol (showing each participant cursor position and color), viewport synchronization through Focus Mode (where the tutor can broadcast their viewport position to all students), remote pen freeze (tutor can disable student drawing), a private scratchpad for tutor notes not visible to students, and live in-classroom polling via Yjs shared data structures.

6.1 Synchronized State

State Type	Yjs Type	Description
Canvas Objects	Y.Array of Y.Map	Per-object CRDT sync (position, style, content)
Multi-Page	ydoc.getMap("meta")	Current page index, total pages
Awareness	Yjs Awareness Protocol	User name, color, role, cursor position
Focus Mode	Awareness State	Tutor viewport (x, y, zoom) broadcast
Pen Freeze	Shared State	Tutor-controlled drawing freeze for students
Scratchpad	Separate Y.Doc	Private tutor scratchpad (not synced)
Live Polls	Y.Map/Y.Array	Real-time polling data via shared structures

6.2 Hocuspocus Server Configuration

The Hocuspocus server runs as a separate mini-service on port 3001 (configurable via `HOCUSPOCUS_PORT`). It connects to the same PostgreSQL database and Supabase instance as the main application. Documents are named using the pattern `room-{roomId}` for simple routing. The server includes a health check endpoint on port+1 (3002) and a background task that expires pending agency invites every 15 minutes.

Configuration	Value	Notes
Port	3001 (configurable)	WebSocket server port
Document Naming	room-{roomId}	Room-based document isolation
Connection Rate Limit	30/min per IP	Prevents connection flooding
Message Rate Limit	100/sec per connection	Prevents message flooding
Max Document Size	10 MB	Prevents memory exhaustion
Health Check Port	PORT + 1	Separate HTTP health server
Invite Cleanup	Every 15 minutes	Batch-expires pending invites

6.3 Hocuspocus Authentication

The `onAuthenticate` hook performs full JWT verification against Supabase Auth. It validates the token and `userId` parameters against `supabaseAdmin.auth.getUser()`, then checks room access by verifying that the room exists, is active, and the user is either the tutor, a registered participant, or an agency owner of the tutor. Room IDs are validated with a strict alphanumeric regex pattern to prevent injection.

6.4 Client-Side Integration

The `useYjsProvider` custom hook manages the Yjs WebSocket lifecycle, including connection establishment, awareness broadcasting, and graceful degradation when the Hocuspocus server is unavailable. The `useYjsCanvasSync` module handles per-object synchronization between Fabric.js canvas objects and Yjs CRDT shared types. The `useFocusMode` hook implements viewport synchronization by broadcasting the tutor viewport coordinates through the awareness protocol and applying them on student clients.

7. AI System

The AI system provides 24 distinct educational actions powered by Anthropic Claude models. Actions are intelligently routed between Claude 3 Haiku (for fast, inexpensive text generation) and Claude 3.5 Sonnet (for vision-dependent tasks requiring image analysis). Each action has a specific credit cost that is deducted from the user budget, and the system implements soft throttling when users approach their monthly AI spending limits.

7.1 Model Routing

Text AI Actions (routed to Claude 3 Haiku) include all original educational content generation actions plus the 10 enhanced actions. Vision AI Actions (routed to Claude 3.5 Sonnet) include graph plotting from handwritten input, geometric shape recognition, handwriting-to-LaTeX conversion, and diagram generation from descriptions. The `getModelForAction()` function determines the model based on whether the action is in the `TEXT_AI_ACTIONS` or `VISION_AI_ACTIONS` set.

7.2 Text AI Actions (Claude 3 Haiku)

Action	Credits	Description
QUIZ	1	Generate quiz questions on specified topic
SUMMARY	1	Summarize lesson content
GRAMMAR	1	Grammar check and corrections
OUTLINE	1	Generate content outline
VOCAB_QUIZ	1	Vocabulary quiz generation
PHONICS_HELPER	1	Phonics assistance for early readers
TIMELINE_GENERATOR	1	Timeline creation for history topics
CONCEPT_SUMMARIZER	1	Concept summarization
CHEMICAL_BALANCER	1	Balance chemical equations
LAB_SUMMARY	1	Lab report summary generation
WORKSHEET	2	Generate practice worksheets
LESSON_PLAN	5	Full lesson plan with objectives, materials, activities (Pro+)
DIFFERENTIATED_INSTRUCTION	5	3-tier differentiated instruction strategies (Pro+)
FORMATIVE_ASSESSMENT	5	5 MCQ + short answer + extended response assessment (Pro+)
RUBRIC_GENERATOR	5	4-level, 4-6 criteria rubric (Pro+)
STUDENT_FEEDBACK	3	Personalized performance feedback (Pro+)
CONCEPT_EXPLAINER	3	Analogy-based concept explanation (Pro+)

Action	Credits	Description
STEP_BY_STEP_SOLVER	3	Step-by-step math solution (Pro+)
FLASHCARD_GENERATOR	3	10 flashcards with difficulty ratings (Pro+)
WORD_PROBLEM_BUILDER	5	5 real-world word problems with MCQ (Pro+)
ANNOTATION_HELPER	3	Text annotation with main idea and vocabulary (Pro+)

7.3 Vision AI Actions (Claude 3.5 Sonnet)

Action	Credits	Description
PLOT_GRAPH	3	Smart graph plotting from handwritten input
PERFECT_SHAPE	2	Perfect geometric shape recognition and cleanup
HANDWRITING_TO_MATH	3	Convert handwritten math to LaTeX notation
DIAGRAM_GENERATOR	3	Generate diagrams from text descriptions

7.4 Credit System

Each AI action costs between 1 and 5 credits. Credit budgets reset weekly (FREE tier) or monthly (PRO and Agency tiers). The system tracks both credit usage and estimated cost in cents. A soft throttle mechanism activates when users approach their monthly AI budget threshold: PRO users are throttled at \$3.00/month (300 cents) and Agency sub-tutors at \$15.00/month (1500 cents). The throttle does not block requests but serves as a warning to prevent runaway costs.

7.5 Prompt Sanitization

The `sanitizePrompt()` function truncates input to 50,000 characters and scans against 10 injection patterns using regex matching. Detected patterns include attempts to ignore previous instructions, change the AI role, execute jailbreak attacks, or extract the system prompt. Matches are replaced with [FILTERED] to preserve context while neutralizing the attack vector. The default system prompt for text actions establishes the AI as a K-12 educational assistant that does not give direct answers to students.

8. Pricing and Tier System

The pricing model supports four tiers with distinct feature sets, usage limits, and billing structures. Feature gating is enforced server-side in API route handlers using the `hasFeature()` function from the `usage.ts` module, which maps legacy AGENCY tier to AGENCY_STANDARD for backward compatibility.

8.1 Pricing Plans

Tier	Monthly	Annual	Per Hour	Max Sub-Tutors
FREE	\$0	\$0	N/A	0
PRO	\$10	\$96	N/A	0
AGENCY_STANDARD	\$39	\$390	\$3.00	5
AGENCY_PREMIUM	\$79	\$790	\$2.00	Unlimited

8.2 Usage Limits

Limit	FREE	PRO	AGENCY_STANDARD	AGENCY_PREMIUM
Max Active Rooms	1	Unlimited	Unlimited	Unlimited
Video Minutes	120/week	Unlimited	Unlimited	Unlimited
AI Credits	10/week	500/month	5,000/month	10,000/month
Recordings	0	2/month	Unlimited	Unlimited
AI Budget Soft Throttle	N/A	\$3/month	\$15/month	\$15/month

8.3 Feature Gating Matrix

Feature	FREE	PRO	AGENCY
Uploads	No	Yes	Yes
Save/Load Boards	No	Yes	Yes
Templates	No	Yes	Yes
PDF Download	No	Yes	Yes
GeoGebra Graphing	No	Yes	Yes
Shape Perfect (AI)	No	Yes	Yes

Feature	FREE	PRO	AGENCY
Mathpix Integration	No	Yes	Yes
Basic AI Tools	Yes	Yes	Yes
Enhanced AI Tools	No	Yes	Yes
Recordings	No	Yes (2/mo)	Yes (Unlimited)
White Label Branding	No	No	Yes
Admin Dashboard	No	No	Yes

8.4 Credit Packs

Agencies can purchase prepaid credit packs for hourly usage at discounted rates. This provides flexibility for agencies with variable tutoring loads. Credit packs are tracked in the CreditPack model with hours remaining decremented as lessons are conducted.

Hours	Price	Effective Rate
20 hours	\$50	\$2.50/hr
50 hours	\$100	\$2.00/hr

9. UI Components Inventory

The application comprises over 75 custom React components organized by functionality. Components follow the single-responsibility principle and use a combination of React Server Components and Client Components as appropriate. Heavy components (GeoGebra, LiveKit video, AI panels) are lazy-loaded via dynamic imports to maintain sub-1.5-second initial page load times.

9.1 Canvas and Whiteboard Components

Component	Purpose
Whiteboard.tsx	Main orchestrator integrating all canvas sub-components
FabricCanvas/index.tsx	Fabric.js canvas with Yjs CRDT sync, multi-page, color-blind support
FabricCanvas/hooks.ts	Canvas tool hooks (drawing, shapes, text, eraser, pan/zoom, undo/redo)
useYjsCanvasSync.ts	Per-object CRDT synchronization between Fabric.js and Yjs

Component	Purpose
Toolbar.tsx	Dynamic subject toolkit switcher with AI tools per subject
PageSidebar.tsx	Multi-page sidebar navigation with thumbnail previews
SessionTimer.tsx	Lesson duration timer (tutor only)
SessionControls.tsx	Focus mode, pen freeze, scratchpad, accessibility toggles
FileAttachmentsBar.tsx	File upload/attachment bar for lesson resources
ImageCompressor.ts	Client-side image compression before upload
PresenceIndicator.tsx	Real-time participant presence indicator with colors
LivePollPanel.tsx	Live in-classroom polling via Yjs shared data
QuestionBankPanel.tsx	Cloud question bank browser with add-to-canvas functionality
ManipulativePanel.tsx	Pre-built math manipulatives library (fraction bars, etc.)
ManipulativeCreator.tsx	AI-powered prompt-to-manipulative generation
NotesAutoGenerator.tsx	Auto-generated lesson notes from canvas content
VideoLimitBanner.tsx	Soft-stop video usage limit warning banner

9.2 AI Feature Components

Component	Purpose
AIControlPanel.tsx	AI features toggle panel organized by subject groups
QuizGenerator.tsx	AI quiz generation with public/private question split
WorksheetGenerator.tsx	AI worksheet generation with configurable options
AnswerKeyModal.tsx	Answer key viewer modal (tutor only)
GeoGebraPanel.tsx	GeoGebra interactive graphing panel (Pro+)
LazyGeoGebraPanel.tsx	Lazy-loaded GeoGebra wrapper for performance

9.3 Dashboard Components

Component	Purpose
DashboardPage.tsx	Main authenticated dashboard with sidebar navigation
MyRoomsPanel.tsx	Active and past lesson rooms list
AnalyticsPanel.tsx	Usage analytics and insights with charts

Component	Purpose
SchedulePanel.tsx	Lesson scheduling calendar view
RecordingsPanel.tsx	Lesson recordings library with playback
TemplatesPanel.tsx	Saved board templates management
SavedBoardsPanel.tsx	Saved board snapshots gallery
BillingPanel.tsx	Subscription and payment management
AgencyAdminPanel.tsx	Agency management (sub-tutors, students, analytics)
StudentManagementPanel.tsx	Student roster management with bulk import
StudentProgressPanel.tsx	Individual student progress tracking
HomeworkPanel.tsx	Homework assignment creation and grading
LessonNotesPanel.tsx	Post-lesson notes and feedback management
ResourceLibraryPanel.tsx	Shared agency resource library
InvoicePanel.tsx	Invoice management and creation
OnboardingWizard.tsx	First-time tutor onboarding flow
StudentDashboard.tsx	Student-specific dashboard view

9.4 Other Key Components

Component	Purpose
LandingPage.tsx	Full marketing page with auth, pricing, features, FAQ
AuthGate.tsx	Client-side auth routing (landing vs dashboard)
WaitingRoom.tsx	Student waiting room shown before tutor joins
NameEntryModal.tsx	Student name entry on room join
PipVideoPanel.tsx	Picture-in-picture video panel
RecordButton.tsx	Lesson recording start/stop button
BrandedHeader.tsx	Custom branded header (agency logo/color)
PaywallModal.tsx	Premium feature paywall modal
UsageBar.tsx	Usage tracking bar (AI credits, video, recordings)
AdminPanel.tsx	Admin dashboard (users, rooms, subscriptions, audit)
MathToolkit.tsx	Math-specific shapes and tools (protractor, ruler)
ScienceToolkit.tsx	Science-specific shapes and tools

Component	Purpose
LanguageToolkit.tsx	Language arts tools
ErrorBoundary.tsx	React error boundary component

10. Custom Hooks

The application uses 9 custom React hooks that encapsulate complex stateful logic for reuse across components. These hooks manage WebSocket connections, video rooms, accessibility modes, and billing state.

Hook	Purpose
useYjsProvider	Yjs + Hocuspocus WebSocket lifecycle management, awareness broadcasting, and graceful degradation
useLiveKitRoom	LiveKit room connection, track management, and heartbeat reporting for video usage tracking
useFocusMode	Tutor-to-student viewport synchronization via Yjs awareness protocol
useCredits	Real-time credit and usage tracking for billing UI display
useAccessibility	Apply color-blind and accessibility modes via data attributes on DOM elements
useToast	Toast notification helper wrapping sonner library
useTheme	Dark/light theme toggle with persistence
use-mobile	Mobile viewport detection for responsive layout adjustments

11. Library Modules

The application includes 20+ library modules in the src/lib directory that provide shared functionality across the application. These modules handle authentication, database access, AI integration, payment processing, rate limiting, validation, audit logging, canvas operations, and more. Each module exports specific functions and types that are consumed by API routes, server components, and client components.

Module	Key Exports	Purpose
auth.ts	verifyAuth(), requireAuth(), requireAdmin()	JWT token verification and auth enforcement
supabase.ts	createBrowserClient(), createServerClient()	Supabase client factory for browser and server
db.ts	prisma singleton	Prisma client singleton for database access

Module	Key Exports	Purpose
ai.ts	callTextAI(), callVisionAI(), getModelForAction()	Anthropic Claude integration with model routing
ai-enhancements.ts	AI_PROMPT_TEMPLATES, buildEnhancedSystemPrompt()	Enhanced AI action prompt templates
stripe.ts	Stripe client, webhook verification	Stripe payment processing setup
stripe-billing.ts	Metered billing, checkout sessions	Hourly billing and subscription management
usage.ts	hasFeature(), checkAICreditLimit(), incrementAICredits()	Tier limits, feature flags, usage tracking
rate-limit.ts	checkRateLimit(), getRateLimitCategory()	Multi-tier rate limiting with Redis/memory
validations.ts	Zod schemas, validateInput()	Centralized input validation for all endpoints
roles.ts	hasPermission(), getUserRole()	Role-based access control helpers
audit.ts	logAudit()	Audit logging for admin actions
auth-fetch.ts	authFetch()	Authenticated fetch wrapper with JWT headers
fingerprint.ts	getFingerprint()	Browser fingerprinting for anonymous students
canvas-export.ts	exportCanvasPNG(), exportCanvasPDF()	Canvas export to image and PDF formats
canvas-to-notes.ts	generateNotesFromCanvas()	Auto-generate lesson notes from canvas content
webhook-dispatcher.ts	dispatchWebhook()	Outbound webhook dispatch with HMAC signing
livekit.ts	LiveKit client config	LiveKit room connection configuration
katex.ts	renderKaTeX()	KaTeX rendering utilities for math formulas
subject-meta.ts	Subject metadata	Subject icons, colors, descriptions

12. Page Routes

The application uses Next.js App Router with file-based routing. The root page (/) serves as both the landing page for unauthenticated users and the dashboard for authenticated users, controlled by the AuthGate component.

Route	Component	Purpose
/	AuthGate -> LandingPage / DashboardPage	Root: marketing for guests, dashboard for users
/dashboard	Redirect to /	Friendly redirect
/room/[roomId]	RoomPage -> Whiteboard	Main whiteboard room with collaboration

Route	Component	Purpose
/invite/[code]	Invite acceptance page	Sub-tutor invite acceptance flow
/parent/[token]	Parent portal page	Parent view of student progress
/terms	Terms of service	Legal page
/privacy	Privacy policy	Legal page
/contact	Contact page	Support contact
/cookies	Cookie policy	Legal page
/refund	Refund policy	Legal page
not-found	Custom 404 page	Not found handler

13. Deployment Infrastructure

The application supports two primary deployment strategies: Vercel for managed cloud deployment and Docker for self-hosted deployments. Both strategies are production-ready with health checks, security hardening, and resource limits.

13.1 Docker Stack

The Docker Compose configuration defines a three-service stack: Next.js application, Hocuspocus WebSocket server, and Caddy reverse proxy. All services run as non-root users with read-only filesystems, tmpfs mounts for temporary files, and strict resource limits. The Hocuspocus server is only accessible within the Docker network (no port mapping to the host), ensuring that WebSocket connections must pass through the Caddy proxy.

Service	Image / Build	Ports	Key Config
app (Next.js)	Dockerfile (multi-stage)	3000:3000	2 CPUs, 2GB RAM, 256 PIDs, read_only
hocuspocus	Dockerfile.hocuspocus	Internal only	1 CPU, 1GB RAM, 128 PIDs, read_only
caddy (proxy)	caddy:2-alpine	80:80, 443:443	0.5 CPU, 512MB RAM, TLS termination

13.2 Dockerfile (Multi-Stage)

The main Dockerfile uses a three-stage build process. Stage 1 (deps) installs dependencies with Bun or npm and generates the Prisma client. Stage 2 (builder) compiles the Next.js application with standalone output mode. Stage 3 (runner) creates a non-root appuser (uid 1001), copies the standalone server output, static files, public directory, Prisma directory, and Caddyfile. The container runs on port 3000 with a health check against /api/health every 30 seconds.

13.3 Environment Variables

The application requires the following environment variables for deployment. These should be configured in a `.env.local` file for development or in the deployment environment variables for production.

Variable	Purpose	Required
DATABASE_URL	PostgreSQL connection string (Supabase)	Yes
DIRECT_URL	Direct database connection (Supabase pooling)	Yes
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL	Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous key	Yes
SUPABASE_SERVICE_ROLE_KEY	Supabase admin service role key	Yes
ANTHROPIC_API_KEY	Anthropic Claude API key	Yes
STRIPE_SECRET_KEY	Stripe secret key	Yes
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret	Yes
STRIPE_AGENCY_STD_PRICE_ID	Stripe price ID for Agency Standard	Yes
STRIPE_AGENCY_PREMIUM_PRICE_ID	Stripe price ID for Agency Premium	Yes
LIVEKIT_URL	LiveKit server URL	Yes
LIVEKIT_API_KEY	LiveKit API key	Yes
LIVEKIT_API_SECRET	LiveKit API secret	Yes
NEXT_PUBLIC_HOCUSPOCUS_URL	Hocuspocus WebSocket URL	Yes
UPSTASH_REDIS_REST_URL	Upstash Redis URL (optional)	No
UPSTASH_REDIS_REST_TOKEN	Upstash Redis token (optional)	No
NEXT_PUBLIC_MAIN_DOMAIN	Main domain for custom domain routing	Yes

14. Webhook System

The application supports outbound webhook notifications that allow users to integrate Superboard events with external systems. Users can register webhook endpoints with a list of event types to subscribe to and an optional HMAC secret for signature verification. The webhook dispatcher includes SSRF (Server-Side Request Forgery) protection by blocking private IP addresses and resolving hostnames to verify they do not point to internal

network resources.

Each webhook dispatch fires asynchronously (fire-and-forget) and includes an HMAC-SHA256 signature in the X-Webhook-Signature header (format: sha256=hex) along with the event type in X-Webhook-Event. Errors are logged but never block the main operation. Webhook URLs must use HTTPS, and DNS resolution is performed to verify that the target is not a private IP address.

15. Audit System

The audit system provides a comprehensive trail of all administrative actions for compliance and accountability. The `logAudit()` function creates `AuditLog` entries with the admin ID, action type, target information, and optional metadata. Audit actions cover user management (create, delete, tier change, status change, admin toggle, password reset), room operations (close, open, delete, force end), subscription management (override, cancel, extend), platform configuration changes (maintenance toggle, announcement change), and bulk operations.

The audit function is designed to be non-blocking: errors in audit logging are caught and logged to the console but never propagate to the caller. This ensures that audit failures do not disrupt the main application operation. Metadata is stored as JSON text for flexible schema-less tracking of action-specific details.

16. PWA Support and Branding

The application includes Progressive Web App (PWA) support via a `manifest.json` and service worker (`sw.js`) for offline caching. This enables the application to be installed on mobile devices and desktops for a native app-like experience. The manifest includes the app name, icons, theme color, and display configuration.

White-label branding is available for Agency Premium tier users. This includes custom domain support (via the `customDomain` field on the `User` model), custom logo and colors (`brandingLogoUrl`, `brandingColor`), and agency name display. The middleware detects custom domains by comparing the request host against `NEXT_PUBLIC_MAIN_DOMAIN` and adds query parameters that the frontend uses to apply the branded experience. Custom branded headers (`BrandedHeader` component) and branded PDF exports (`BrandedPdfExport`) ensure a consistent branded experience throughout the application.

17. Data Seeding Scripts

The project includes several database seeding scripts for development and testing purposes. These scripts populate the database with test data including sample users, question banks, and curriculum standards. The scripts are

located in the scripts/ directory and use Prisma client for database operations.

Script	Purpose
scripts/seed.ts	Main database seeder
scripts/seed-questions-bulk.ts	Bulk question bank seeder
scripts/seed-users.ts	Test user creation (admin, tutor, student)
scripts/seed-question-bank.ts	Question bank with standards alignment
scripts/migrate.ts / migrate.js	Database migration scripts
src/data/seed-questions.ts	Question bank seed data

18. Key Architectural Patterns

The application follows several established architectural patterns that ensure scalability, maintainability, and security. Understanding these patterns is essential for any development team building or extending the application.

Pattern	Implementation	Benefit
App Router (Next.js 16)	File-based routing with RSC and client components	Simplified routing, server components for performance
Zustand + TanStack Query	Zustand for client state, TanStack for server state	Clear separation of concerns, automatic cache invalidation
Zod v4 Validation	Centralized schemas in validations.ts	Type-safe input validation with consistent error responses
Prisma ORM	Schema-first database access with migrations	Type-safe queries, automatic migration management
CRDT Collaboration	Yjs for conflict-free real-time editing	No central conflict resolution, works offline, resilient to latency
Lazy Loading	Dynamic imports for heavy components	Sub-1.5s initial load, on-demand loading of GeoGebra, LiveKit, AI panels
Server-Side Authorization	All tier/feature gating in API routes	Security enforced at server level, not client level
Multi-Tenant Agency	Sub-tutors, student rosters, branded experiences	Scalable organizational model with data isolation
Metered Billing	Per-hour tracking via room duration + Stripe	Usage-based pricing aligned with actual tutoring time
Audit Trail	Comprehensive logging of all admin actions	Compliance, accountability, and debugging support